

Data Lakehouse

MIMIC-OMOP

**Paloma Bravo del Cid
Maria Luisa Rodriguez Cabrera
Francisco Rey Garduño**



Master en Ingeniería del Software
Asignatura de Big Data
Universidad de Sevilla

Índice

Data Lakehouse MIMIC-OMOP.....	0
1. Objetivo del trabajo.....	2
2. Tecnología elegida y justificación.....	2
3. Arquitectura y flujo de datos.....	4
4. Desarrollo por Hitos.....	4
4.1. Issue 1 - Configuración de Infraestructura Base.....	4
A. Orquestación de Contenedores (Imagen 1).....	4
B. Gestión de Secretos y Variables de Entorno (Imagen 2).....	4
C. Acceso a las Interfaces de Gestión (Imagen 3 y 4).....	5
4.2. Issue 2 - Orquestación de Cargas Incrementales (Apache Airflow).....	5
A. Pipeline Completado (Grid View) (Imagen 5).....	5
B. Carga de Datos en MinIO (Imagen 6).....	5
4.3. Issue 3 - Desarrollo modelo dbt: person.sql.....	5
Implementación Técnica del Modelo person.sql.....	6
A. Definición de Fuentes (sources.yml).....	6
B. Documentación y Validación de Modelos (schema.yml).....	6
C. Organización y estructura modelos (Imagen 7).....	6
4.4. Issue 4 - Configuración de MinIO y Catálogo Hive.....	7
4.5. Issue 5 - Analítica masiva con Apache Spark.....	7
4.6. Issue 6 - Modelo dbt: visit_occurrence.sql.....	9
4.7 Issue 7 - Simulador de ingesta de datos.....	10
4.8 Issue 8 - Inicialización de dbt y Trino.....	11
4.9 Issue 9 — Pruebas de Calidad y Docs (dbt tests).....	11
4.10 Issue 10 — Desarrollo modelo dbt: condition_occurrence.sql.....	12
5. Dificultades encontradas y soluciones.....	12
Imagen 6: Carga de datos en MinIO.....	16
7. Reparto del trabajo.....	18
8. Conclusiones.....	19

1. Objetivo del trabajo

Este proyecto tiene como objetivo principal el diseñar, desplegar y automatizar una arquitectura de datos moderna basada en el paradigma Data Lakehouse, aplicada a un caso de uso real y crítico dentro de la industria sanitaria: la estandarización y análisis de historiales clínicos electrónicos de forma masiva.

Para ello, el proyecto persigue la transformación de los datos en crudo procedente de la base de datos de UCI MIMIC hacia el modelo de datos clínicos estandarizados de OMOP CDM, simulando un entorno de producción hospitalario con ingesta continua de datos.

Este objetivo se desglosa en los siguientes objetivos específicos:

1. Desacoplamiento de almacenamiento y cómputo: Evolucionar desde las arquitecturas monolíticas tradicionales hacia una infraestructura Cloud-Native basada en el almacenamiento de objetos, garantizando una mayor escalabilidad y reducción de costes operacionales.
2. Implementación de formatos de tabla abierto: aplicar el estándar apache iceberg para dotar al Data Lake de capacidades propias de una base de datos relacional que solucione las carencias del formato parquet tradicional.
3. Gobierno y estandarización del dato: aplicar las mejores practicas de la ingenieria de software a los pipeline mediante el modelado modular, asegurando la integridad y la coherencia clínica de los registros antes de su consumo analítico
4. Orquestación de cargas incrementales: simular la llegada diaria de nuevos registros hospitalarios implementando un flujo de trabajo automatizado que procese únicamente los datos nuevos, optimizando los recursos computacionales frente a los reprocesamientos completos.
5. Democratización de la analítica masiva: exponer la capa de datos estandarizada para que motores de procesamiento distribuido puedan extraer conocimiento médico de alto valor de forma eficiente.

2. Tecnología elegida y justificación

Para alcanzar los objetivos descritos en el apartado anterior, hemos decidido replicar un entorno moderno de datos utilizado empresarialmente. Cada una de las herramientas las hemos seleccionado por su capacidad de integración y su predominancia en el mercado laboral actual.

Para el almacenamiento, en este caso de objetos hemos utilizado MinIO que es la solución de código abierto y de alto rendimiento, que tiene compatibilidad al 100% con el protocolo S3 de Amazon. Esta nos permite desacoplar el almacenamiento del cómputo, además, es muy ligero y gestiona de manera nativa los archivos generados en las distintas capas de nuestra arquitectura.

Para que nuestro Data Lakehouse fuera de verdad un Data Lakehouse operativo, hemos utilizado Apache Iceberg que actúa como una capa de metadatos que permite realizar operaciones transaccionales (ACID) sobre MinIO. La hemos elegido por su neutralidad y por la integración con Trino, además de por su capacidad de Time Travel, que nos permite asegurar la trazabilidad.

Para las consultas distribuidas hemos decidido utilizar Trino que lo hemos usado en clase y es un motor de consultas muy rápido que es capaz de leer directamente desde el almacenamiento de objetos, además destacando su alto rendimiento en concurrencia y la integración nativa con iceberg. Aunque en este caso no utilizamos Hadoop, el servicio de Hive Metastore lo hemos utilizado ya que actúa como índice central que le dice a Trino donde están físicamente los archivos de Iceberg dentro de los bucket de MinIO y cual es su esquema.

Para la parte de transformación desde MIMIC a OMOP, hemos optado por usar dbt ya que es una tecnología que ha revolucionado la ingeniería de datos al permitir escribir transformaciones modulares puramente en SQL. Sus principales ventajas para este proyecto son: la capacidad de materializar modelos directamente como tablas Iceberg, la generación automática de grafos de dependencias y la integración de test de calidad de datos nativos que bloquean el pipeline si los datos clínicos contienen anomalías.

Para la orquestación de pipelines hemos utilizado Apache Airflow que es el líder en la creación de DAGs mediante Python, lo hemos elegido por su capacidad de programar dependencias complejas, manejar reintentos en caso de fallos y gestionar cargas de datos incrementales de manera idempotente.

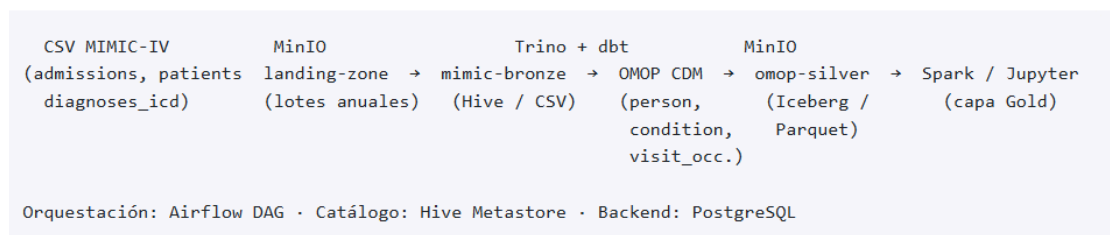
Una vez que los datos están limpios y estandarizados en formato Iceberg, hemos decidido usar Apache Spark para obtener valor analítico de estos datos. Se ha elegido por la capacidad de procesamiento en memoria a gran escala que nos permite hacer cruces estadísticos en cuestión de segundos, demostrando el valor final de la arquitectura.

Por último, para garantizar la reproducibilidad del entorno y facilitar el trabajo colaborativo entre todos nosotros, hemos decidido usar docker para contenerizar toda la arquitectura, aislando las dependencias del sistema operativo base.

3. Arquitectura y flujo de datos

El flujo se divide en tres capas lógicas:

1. **Bronze (Landing):** Datos crudos cargados en MinIO mediante Airflow.
2. **Silver (Staging):** Datos limpiados y tipados.
3. **Gold (OMOP):** Modelos finales listos para analítica.



4. Desarrollo por Hitos

4.1. Issue 1 - Configuración de Infraestructura Base

El objetivo de este primer hito fue el despliegue y orquestación de la infraestructura necesaria para soportar un **Data Lakehouse** médico. Hemos optado por una arquitectura basada en microservicios utilizando Docker Compose, lo que garantiza que todo el stack tecnológico sea reproducible y portable, independientemente del sistema operativo (Windows, macOS o Linux).

La infraestructura se compone de los siguientes contenedores interconectados:

- **Apache Airflow:** Orquestador de los flujos de datos (DAGs).
- **Trino (SQL Engine):** Motor de consultas de alto rendimiento que actúa como interfaz entre el almacenamiento y el usuario.
- **MinIO:** Almacenamiento de objetos compatible con S3, donde reside físicamente el Data Lake.
- **PostgreSQL:** Base de datos relacional utilizada para almacenar los metadatos tanto de Airflow como del Hive Metastore.
- **Hive Metastore:** Catálogo encargado de gestionar la definición de las tablas y sus esquemas sobre el almacenamiento de MinIO.

A. Orquestación de Contenedores (Imagen 1)

Para validar que la infraestructura se ha levantado correctamente, se utiliza el comando `docker ps` o la interfaz de Docker Desktop.

- En la figura se ve las redes internas de Docker permiten la comunicación entre servicios y que los volúmenes de persistencia han sido montados correctamente.

B. Gestión de Secretos y Variables de Entorno (Imagen 2)

Una parte crítica fue la configuración del archivo `.env`

- El uso de este archivo permite separar la configuración de la infraestructura del código fuente, siguiendo las mejores prácticas de seguridad en ingeniería de datos. Se definen las credenciales de MinIO y la `AIRFLOW_FERNET_KEY`

C. Acceso a las Interfaces de Gestión (Imagen 3 y 4)

Se verificó el acceso a las herramientas de administración:

- **MinIO Console (Puerto 9001):** bucket creado.
- **Airflow UI (Puerto 8080/8083):** el orquestador está listo para recibir DAGs.

4.2. Issue 2 - Orquestación de Cargas Incrementales (Apache Airflow)

El objetivo de este issue fue diseñar y ejecutar un pipeline de datos (DAG) que automatizara la ingesta de archivos desde el sistema local hacia la zona **Bronze** del *Lakehouse*. La clave de esta issue fue la **incrementalidad**: el sistema no debe procesar todos los datos siempre, sino solo los nuevos lotes (anuales o diarios) que se depositan en la *Landing Zone*.

El flujo de trabajo se programó en Python utilizando la API de Airflow. Los componentes principales son los siguientes:

- **Ingesta:** Se implementó una lógica donde Airflow detecta archivos CSV en una carpeta local y los transfiere a **MinIO** utilizando el protocolo S3.
- **Tareas definidas en el DAG:**
 1. `subir_lote_minio`: Utiliza un `PythonOperator` para cargar los archivos a los buckets correspondientes.
 2. `mover_a_bronze`: Organiza los archivos dentro de la estructura de carpetas del Lakehouse.
 3. `dbt_run`: Ejecuta las transformaciones SQL una vez los datos están disponibles.
 4. `dbt_test`: Valida que los datos cargados cumplen con las reglas de integridad.

A. Pipeline Completado (Grid View) (Imagen 5)

La prueba definitiva de este bloque de tareas es la visualización del grafo de tareas en verde.

- Cada cuadrado verde representa una tarea que ha finalizado correctamente, confirmando la comunicación entre Airflow y dbt.

B. Carga de Datos en MinIO (Imagen 6)

- Podemos confirmar que la tarea de subida al almacenamiento S3 funciona y que los archivos son accesibles para el motor de consultas Trino.

4.3. Issue 3 - Desarrollo modelo dbt: *person.sql*

El objetivo de este hito fue la creación de la capa **Gold** (OMOP CDM) del Lakehouse. No se trata de una simple copia de datos, sino de un proceso de **ETL Semántico**. Utilizamos **dbt (Data Build Tool)** para transformar las tablas crudas de MIMIC-IV (capa Bronze) en tablas estandarizadas que siguen el vocabulario internacional de salud.

Implementación Técnica del Modelo [person.sql](#)

El modelo [person](#) es la base del estándar OMOP. Se implementó siguiendo estas recomendaciones de eficiencia y calidad:

- **Estrategia Incremental:** importante configurar como `materialized='incremental'` con estrategia `merge`. Esto permite que el pipeline solo procese los nuevos pacientes detectados, optimizando el tiempo de ejecución y el coste computacional.
- **Mapeo Semántico (Concept IDs):** A diferencia de MIMIC, donde el género es un texto ('M', 'F'), OMOP requiere códigos numéricos universales. Se implementó una lógica de `CASE` para asignar los IDs estándar: **8507** (Masculino) y **8532** (Femenino).
- **Lógica Incremental:** Se añadió un filtro Jinja para asegurar que no se dupliquen registros mediante la validación: `WHERE subject_id NOT IN (SELECT person_id FROM {{ this }})`.

La ingeniería de datos moderna además exige que los datos estén documentados y testeados, por tanto procedimos con los siguientes pasos:

A. Definición de Fuentes ([sources.yml](#))

En este archivo hemos definido el contrato con la capa Bronze.

- Se especificó que los datos crudos residen en el catálogo de **Hive** bajo el esquema `mimic_bronze`.
- **Tests Automáticos:** Se añadieron pruebas de integridad directamente en la fuente para detectar errores antes de la transformación:
 - `unique` y `not_null` para identificadores.
 - `accepted_values` para asegurar que el género solo contenga 'M' o 'F'.

B. Documentación y Validación de Modelos ([schema.yml](#))

Aquí definimos las expectativas del modelo final `condition_occurrence` (diagnósticos clínicos).

- Se documentó cada campo para asegurar la trazabilidad.
- Se añadieron tests de integridad referencial y de dominio (ej. validar que el tipo de condición sea siempre el código estándar de EHR **32817**).

C. Organización y estructura modelos (Imagen 7)

- Se observa la separación clara entre `macros`, `models` y `seeds`, siguiendo las mejores prácticas de la industria.
- **Compilación dbt:** Se verificó que dbt genera el SQL correcto para Trino/Iceberg, permitiendo la creación de tablas con metadatos optimizados.

4.4. Issue 4 - Configuración de MinIO y Catálogo Hive

Este Issue agrupa intervenciones sobre la infraestructura del proyecto. Resuelve 2 frentes distintos: la introducción del vocabulario OMOP en la pila de datos, y un saneamiento general de configuración y deudas técnicas acumuladas.

1. Refactor de `upload_to_minio.py`. La versión original subía a `mimic-bronze` con rutas codificadas y no era importable como módulo. La nueva expone la función `upload_lotes_to_landing()`, que recorre la jerarquía `lotes_landing/YYYY/*.csv` y la sube al bucket `landing-zone` preservando la organización por año. El cambio de bucket destino corrige la arquitectura: los datos externos deben entrar primero por la zona de aterrizaje y no directamente en Bronze.

2. Nuevo `upload_vocabulario.py`. Sube `CONCEPT.csv` y `CONCEPT_RELATIONSHIP.csv` al bucket `mimic-vocabulary`, creando una subcarpeta por tabla. Esta separación es necesaria porque Trino, al registrar tablas externas con `external_location`, lee todos los ficheros del directorio bajo el supuesto de que comparten esquema; si ambos ficheros convivieran en la raíz, fallaría la lectura.

3. Integración en `simulador_lotes.py`. Se añade al final del `main()` la llamada a `upload_lotes_to_landing()`, de modo que una sola ejecución del simulador genera los lotes y los sube en una operación atómica. Esto sólo es posible gracias a la refactorización del punto 1, que convirtió el script en módulo importable.

4. Nuevo `init_vocabulario.sql`. Registra el esquema `hive.mimic_vocabulary` y las tablas externas `concept` y `concept_relationship` en Hive Metastore. Todas las columnas se declaran como `VARCHAR` para evitar fallos de parseo en la capa de exposición; las conversiones de tipo se difieren a los modelos dbt mediante `TRY_CAST`. La opción `skip_header_line_count = 1` descarta la línea de cabecera, y el uso de `CREATE ... IF NOT EXISTS` hace el script idempotente.

Decisiones técnicas destacadas:

- Distinción explícita entre *landing zone* y Bronze, alineando la implementación con el patrón de capas del lakehouse.
- Tipos `VARCHAR` en la capa de exposición del vocabulario, posponiendo conversiones a las transformaciones dbt para maximizar la robustez.
- Automatización end-to-end del pipeline de simulación.
- Higiene del repositorio mediante un `.gitignore` que separa código fuente, datos generados y secretos.

4.5. Issue 5 - Analítica masiva con Apache Spark

1. Servicio Spark en `docker-compose`. Se añade un contenedor basado en la imagen `jupyter/pyspark-notebook:latest`, que expone JupyterLab en el puerto 8888 y la Spark UI en el 4040. El volumen `./notebooks:/home/jovyan/work` persiste los notebooks fuera del ciclo de vida del contenedor, y la adscripción a `trabajodb_network` permite resolver por nombre de servicio el resto de componentes.

6. Corrección de credenciales de Trino. 3 ficheros de configuración contenían marcadores sin sustituir ({MINIO_ROOT_USER}, {MINIO_ROOT_PASSWORD}). Trino no expande variables de entorno en estos ficheros, por lo que los marcadores se interpretaban literalmente y provocaban fallos de autenticación silenciosos contra MinIO. Se sustituyen por los valores reales `minioadmin`.

7. .gitignore reescrito. Se añaden exclusiones específicas: `*.csv` y `data/` (datos MIMIC no versionables por tamaño y licencia), `docker/.env` y configuraciones de Trino (credenciales), `datos_simulados/lotas_landing/` (datos generados localmente), `docker/notebooks/*.ipynb` (notebooks de usuario) y los directorios de dbt (`target/`, `dbt_packages/`, `logs/`).

Una vez consolidada la capa silver en formato Iceberg sobre MinIO, surge la necesidad de explotar analíticamente los datos a una escala que excede las capacidades de la librería pandas. Apache Spark se selecciona por su integración nativa con Iceberg, su compatibilidad con almacenamiento S3 vía el conector `s3a` y su capacidad para ejecutar consultas distribuidas. El resultado es el notebook `docker/notebooks/MIMIC-OMOP_Analytics.ipynb`, organizado en cuatro pasos secuenciales.

Paso 1 — Descarga de JARs. Se descargan automáticamente desde Maven Central tres dependencias necesarias para que Spark trabaje contra Iceberg sobre MinIO: `iceberg-spark-runtime-3.5_2.12-1.6.1.jar` (runtime de Iceberg para Spark 3.5), `hadoop-aws-3.3.4.jar` (conector que implementa el sistema de ficheros `s3a://`) y `aws-java-sdk-bundle-1.12.262.jar` (SDK de AWS, dependencia transitiva). Los JARs se persisten en el volumen Docker `/home/jovyan/work/jars`, de modo que la descarga (≈200 MB) sólo se realiza la primera vez. Adicionalmente se instala `pyiceberg` vía pip.

Paso 2 — Inicialización de SparkSession. La construcción de la `SparkSession` concentra la práctica totalidad de la complejidad de configuración y se agrupa conceptualmente en tres bloques. El bloque del catálogo Iceberg registra un catálogo lógico llamado `iceberg` cuyo backend es Hive Metastore (accesible vía Thrift en `hive-metastore:9083`) y cuyo warehouse apunta a `s3a://omop-silver`. El bloque de acceso a MinIO configura el conector `s3a` con `path.style.access=true` (MinIO no soporta el direccionamiento virtual-hosted style de AWS S3) y declara explícitamente el implementador `S3AFileSystem` y el proveedor de credenciales `SimpleAWSCredentialsProvider`. El bloque de credenciales lee las variables de entorno `MINIO_ACCESS_KEY` y `MINIO_SECRET_KEY` con fallback a `minioadmin`, evitando que las credenciales queden incrustadas en el notebook versionado.

Antes de invocar `getOrCreate()`, el notebook detiene cualquier sesión previa mediante `existing.stop()`, ya que `getOrCreate()` se comporta como un singleton: una segunda llamada con configuración distinta devuelve la sesión antigua sin lanzar advertencia.

Paso 3 — Lectura de tablas Iceberg. Mediante `spark.read.table("iceberg.omop.person")` y `spark.read.table("iceberg.omop.condition_ocurrence")`. El identificador sigue el patrón `<catálogo>.<schema>.<tabla>`. Cabe destacar la grafía `condition_ocurrence` (con una sola 'r'), errata heredada de la ingesta original que se respeta para coincidir con el nombre real persistido en el Hive Metastore.

Paso 4 — Consultas analíticas.

Query 1 — Top 10 diagnósticos por género. Cruza condiciones con personas mediante `person_id` y aplica el patrón Top-N por grupo con funciones de ventana: tras agregar las ocurrencias de cada par (género, diagnóstico), la función `rank()` se aplica sobre una ventana particionada por género y ordenada por frecuencia descendente, filtrando después por `rank <= 10`. Como muestra, el diagnóstico 4019 (hipertensión esencial en ICD-9) lidera en mujeres con 18 ocurrencias.

Query 2 — Comorbilidades más frecuentes. Identifica pares de diagnósticos coocurrentes en la misma visita mediante un self-join sobre `condition_occurrence`, vinculado por `visit_occurrence_id`. La cláusula adicional `c1.condition_source_value < c2.condition_source_value` cumple una doble función: descarta los emparejamientos triviales de una fila consigo misma y evita que el par (A, B) se cuente dos veces como (A, B) y (B, A). Esta forma de ordenar los datos alfabéticamente es mejor que usar un `DISTINCT` después, porque hace que el `join` intermedio tenga muchos menos registros. El par más frecuente es (2724, 4019), ya que aparece junto 4 veces, lo que sugiere una posible relación entre la hiperlipidemia y la hipertensión.

Decisiones técnicas destacadas:

- Destrucción explícita de la SparkSession previa (`existing.stop()`) para evitar conflictos derivados del comportamiento singleton de `getOrCreate()`.
- Persistencia de los JARs en volumen Docker para no re-descargarlos en cada reinicio.
- Lectura de credenciales desde variables de entorno, alineada con el resto de la pila.

4.6. Issue 6 - Modelo dbt: visit_occurrence.sql

La tabla `visit_occurrence` es la entidad pivote del modelo OMOP: la mayoría de eventos clínicos (diagnósticos, prescripciones, mediciones) se contextualizan en relación con la visita en la que se produjeron. Este Issue implementa el modelo dbt que transforma la tabla `admissions` de MIMIC en `visit_occurrence` de OMOP.

Configuración del modelo. El modelo se actualiza de forma incremental usando una estrategia `merge` y la clave única `visit_occurrence_id`. Esto significa que, en lugar de volver a procesar todos los datos hospitalarios cada vez, sólo se añaden o actualizan los registros nuevos o modificados, porque los datos históricos de admisiones normalmente no cambian. Las admisiones históricas son inmutables y reprocesarlas en cada ejecución sería innecesariamente costoso. La estrategia `merge` aprovecha las capacidades transaccionales de Iceberg para fusionar registros nuevos con los existentes, permitiendo actualizar visitas previamente cargadas (por ejemplo, cuando se rellena `dischtime` al recibir el alta).

Lógica de transformación. El modelo se organiza en dos CTEs consecutivas. La primera, `raw_admissions`, obtiene los datos de `mimic.admissions` y, en las ejecuciones incrementales, solo carga las admisiones nuevas, es decir, aquellas cuya fecha de ingreso (`admittime`) es posterior a la última fecha ya procesada. Para ello se utiliza la macro `{{ this }}`, que permite consultar el propio modelo.

La segunda CTE, `transformed`, se encarga de seleccionar y transformar los datos. La función `TRY_CAST(date_parse(...) AS DATE)` convierte las fechas de forma segura: si algún valor tiene un formato incorrecto, devuelve `NULL` en lugar de hacer fallar todo el modelo.

Mapeo de campos MIMIC → OMOP:

Campo MIMIC	Campo OMOP	Transformación
<code>hadm_id</code>	<code>visit_occurrence_id</code>	directo
<code>subject_id</code>	<code>person_id</code>	directo (cast a VARCHAR para el join)
<code>admission_type</code>	<code>visit_concept_id</code>	EMERGENCY/URGENT → 9203, resto → 9201
<code>admittime</code>	<code>visit_start_date</code>	<code>date_parse</code> + <code>TRY_CAST</code> a DATE
<code>dischtime</code>	<code>visit_end_date</code>	<code>date_parse</code> + <code>TRY_CAST</code> a DATE (nullable)
—	<code>visit_type_concept_id</code>	constante 32817 (<i>EHR encounter</i>)
<code>admission_type</code>	<code>visit_source_value</code>	valor original sin transformar
—	<code>visit_source_concept_id</code>	constante 0 (sin mapeo directo)

La transformación de `admission_type` a `visit_concept_id` mediante `CASE` mapea al vocabulario controlado de OMOP: 9203 (*Emergency Room Visit*) y 9201 (*Inpatient Visit*). La cláusula `ELSE 9201` actúa como mapeo por defecto, garantizando que la columna nunca quede vacía.

Integridad referencial. El `INNER JOIN` con `{{ ref('person') }}` garantiza que sólo se cargan visitas de pacientes ya existentes en la tabla `person`, evitando referencias huérfanas en la capa Silver. El uso de `{{ ref(...) }}` (en lugar de un nombre literal) permite a dbt construir el grafo de dependencias y materializar `person` antes que `visit_occurrence`. El doble `CAST AS VARCHAR` normaliza tipos y evita fallos de incompatibilidad en el join.

Decisiones técnicas destacadas:

- Actualización incremental mediante estrategia *merge*, para mejorar el rendimiento manteniendo la consistencia de los datos.
- Uso de `INNER JOIN` con la tabla `person` para asegurar que solo se incluyan registros válidos relacionados con pacientes existentes.
- Uso constante de `TRY_CAST` para evitar errores cuando existen datos con formatos incorrectos.

4.7 Issue 7 - Simulador de ingesta de datos

En esta tarea se ha desarrollado el script `'simulador_lotes.py'` con pandas que trocea los tres CSV principales de MIMIC-IV en lotes anuales. La partición no es aleatoria, cada tabla se asigna al año que le corresponde según los datos clínicos reales. (Se ha decidido hacer por años ya que al tener datos de muestras por día tendríamos muy poco registros para procesar a la vez por lo que se decidió hacerlo por año para que el número de registros fuera un poco mayor)

- `admissions.csv`: se ha particionado por el año extraído de `admittime`
- `diagnoses_icd.csv`: hereda el año de su ingreso vía join con `admissions`.
- `patients.csv`: cada paciente aparece en el año de su primera admisión.

El resultado obtenido es un conjunto de carpetas dentro de `lotes_landing/YYYY/` con los tres CSV por año. Al terminar, este script llama automáticamente a `upload_to_minio.py` para subir los lotes al bucket `landing-zone` de MinIO. Las credenciales se leen de variables de entorno para no hardcodearlas.

4.8 Issue 8 - Inicialización de dbt y Trino

Se configuró el proyecto dbt apuntando al catálogo iceberg de Trino local mediante `profiles.yml`. La configuración global de `dbt_project.yml` define que todos los modelos se materializan en formato Iceberg sobre Parquet:

models:

mimicToOmop:

+materialized: table

+table_type: iceberg

+format: parquet

Cada modelo sobreescribe esto individualmente con `materialized = 'incremental'` e `incremental_strategy = 'merge'` para garantizar la idempotencia en reprocesados.

Se definió `sources.yml` con las tres fuentes de MIMIC (`patients`, `admissions`, `diagnoses_icd`) apuntando al schema `mimic_bronze` en el catálogo Hive, y el vocabulario OMOP (`concept`, `concept_relationship`) en el schema `vocabulary`. Incluye tests de calidad en las propias fuentes: `unique/not_null` en claves primarias, `accepted_values` en `gender` (M/F), `icd_version` (9/10) y `hospital_expire_flag` (0/1).

4.9 Issue 9 — Pruebas de Calidad y Docs (dbt tests)

Se implementaron tests en dos niveles:

Tests de esquema (en `schema.yml`) sobre los modelos OMOP resultantes:

- `condition_occurrence`: `unique/not_null` en `condition_occurrence_id`, `not_null` en todas las claves foráneas, `accepted_values`: [32817] en `condition_type_concept_id`
- `visit_occurrence`: `unique/not_null` en `visit_occurrence_id`, `accepted_values`: [9201, 9203] en `visit_concept_id`

Tests singulares (en tests/):

- *assert_dischtime_after_admittime.sql*: verifica que ninguna fecha de alta sea anterior a la de ingreso en la tabla fuente admissions
- *assert_condition_dates_coherent.sql*: misma verificación sobre condition_occurrence
- *assert_condition_person_exists.sql*: detecta diagnósticos sin paciente asociado en person (WARN esperado si los lotes no se han cargado todos)
- *assert_condition_visit_exists.sql*: detecta diagnósticos sin visita asociada en visit_occurrence
- *assert_condition_concept_mapping_rate.sql*: alerta si el % de mapeo ICD→OMOP cae por debajo del umbral esperado

Resultado de ejecución real: 33 PASS, 1 WARN, 0 ERROR.

4.10 Issue 10 — Desarrollo modelo dbt: condition_occurrence.sql

El modelo *models/condition_occurrence.sql* transforma los diagnósticos ICD de MIMIC al estándar OMOP *condition_occurrence*. Las decisiones técnicas clave:

- **Clave surrogada:** *condition_occurrence_id* se genera con *dbt_utils.generate_surrogate_key(['subject_id', 'hadm_id', 'seq_num'])* — un hash MD5 determinista que garantiza unicidad sin depender de secuencias numéricas y permite merge incremental sin duplicados.
- **Mapeo de vocabulario:** doble join con las tablas *concept* y *concept_relationship* (vocabulario OMOP descargado de Athena OHDSI). Se usa LEFT JOIN para no perder diagnósticos sin mapeo, asignando *concept_id* = 0 cuando no existe correspondencia.
- **Incrementalidad:** filtra solo los ingresos posteriores a *MAX(condition_start_date)* de la tabla destino, procesando un lote anual por ejecución del DAG.

5. Dificultades encontradas y soluciones

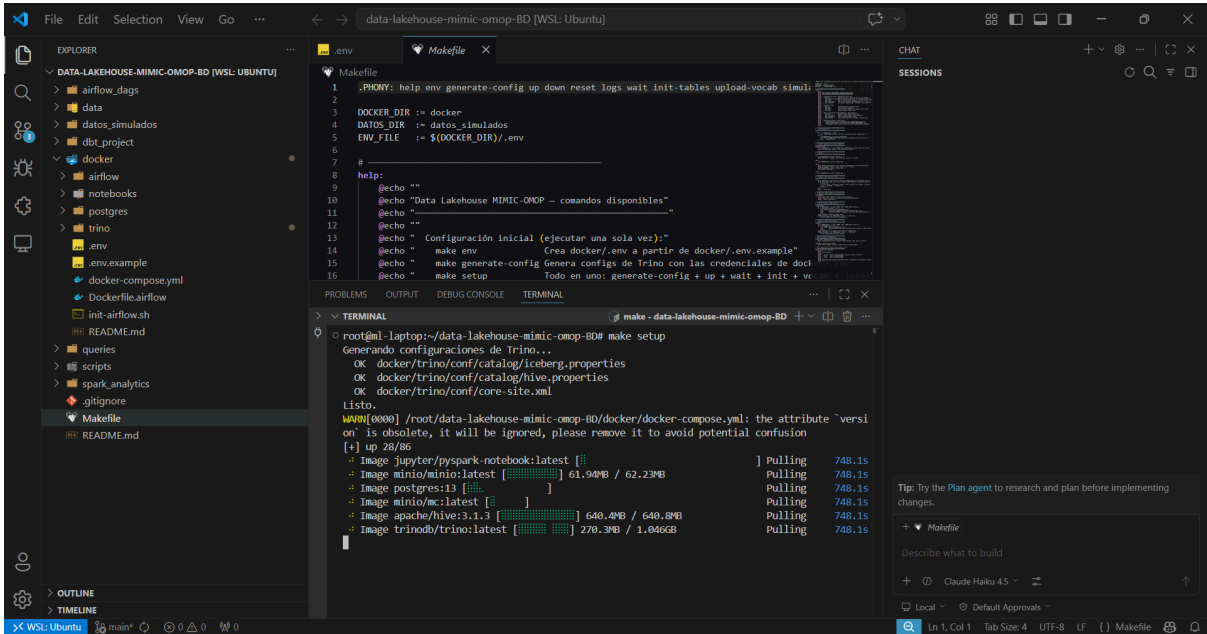
1. **Error de autenticación S3 en tres capas distintas:** este error ha sido uno de los que más dolores de cabeza nos ha dado ya que al ejecutar el DAG completo, aparecían tres errores distintos en tres tareas distintas, cada uno con un mensaje diferente, aunque todos tenían la misma causa raíz: las credenciales de MinIO eran incorrectas. Trino accede a MinIO desde tres rutas de código independiente, cada una con su propio fichero de configuración (*iceberg.properties*, *hive.properties*, *core-site.xml*), estos estaban configurados con una contraseña estática que no era la que estaba en el *.env*. El síntoma cambiaba en cada tarea, lo que dificultó identificar que el origen era el mismo. Una vez que localizamos la causa, la solución fue poner variables donde antes teníamos las contraseñas estáticas para que las cogiera del archivo *.env*, evitando que este problema pueda repetirse.
2. **Hive CSV SerDe sólo admite VARCHAR:** al intentar mejorar el tipado de las tablas bronze declarando columnas como *concept_id* como INTEGER en el SQL de inicialización, Trino devolvió: 'Hive CSV storage format only supports VARCHAR (unbounded). Unsupported columns: concept_id integer'. El formato CSV de Hive no tiene tipado real a nivel de almacenamiento, todas las columnas deben declararse como VARCHAR independientemente de su contenido. La solución fue revertir las columnas al tipo VARCHAR y delegar esto a los

modelos dbt mediante TRY_CAST, que además gestiona valores nulos o malformados sin lanzar excepciones.

3. **shutil.rmtree falla en Docker con bind mounts de macOS:** La tarea del DAG que borra el lote procesado localmente usaba shutil.rmtree(). En Python 3.11 esta función internamente usa _rmtree_safe_fd, una implementación que requiere el syscall dir_fd para abrir directorios de forma segura. Este syscall no está soportado por el sistema de ficheros virtual que Docker Desktop crea en macOS para los bind mounts. El resultado era FileNotFoundError al intentar borrar admissions.csv dentro del lote, dejando el directorio en un estado inconsistente. La solución fue sustituir shutil.rmtree(ruta_lote) por subprocess.run(['rm', '-rf', ruta_lote], check=True), que delega el borrado al proceso rm del sistema operativo del contenedor (Linux), donde el syscall sí está disponible.
4. **Esquemas MIMIC desactualizados (III vs IV)** Las tablas patients, admissions y diagnoses_icd tenían columnas de MIMIC-III que no coincidían con los CSVs reales (MIMIC-IV). Solución: reescribir los CREATE TABLE con el esquema correcto (anchor_age/anchor_year, race, icd_code+icd_version).
5. **Protocolo S3 incorrecto (s3a:// vs s3://)** Trino con conector Hive nativo (fs.native-s3.enabled=true) requiere s3://, no el protocolo Hadoop. Solución: cambiar todas las rutas en init_mimic_bronze.sql.
6. **concept_id como INTEGER rompe el vocabulario OMOP** Los CSVs de Athena OHDSI contienen valores no puramente numéricos. Solución: cambiar concept_id, concept_id_1 y concept_id_2 a VARCHAR.
7. **Configuración doble s3a:// y s3:// en Spark** Iceberg y Hadoop usan protocolos distintos; con uno solo fallaban operaciones. Solución: configurar ambos (fs.s3a.* y fs.s3.*) en la SparkSession.
8. **TRY_CAST(date_parse(...)) para fechas MIMIC** CAST(admittime AS DATE) falla por el formato YYYY-MM-DD HH:MM:SS. Solución: aplicar el patrón ya depurado para visit_start_date y visit_end_date.

6. Resultados y evidencias de funcionamiento

Imagen 1: Configuración global del proyecto



☐	docker	-	-	-	5.79%	23 minutes ago		:	
☐	minio	bea7ca9a99d3	minio/minio:latest	9000:9000 Show all ports (2)	0.04%	23 minutes ago		:	
☐	postgres	a369747e2575	postgres:13	5435:5432 Show all ports (2)	0.71%	23 minutes ago		:	
☐	spark	c3b5f3ca7417	jupyter/pyspark-notebook	4040:4040 Show all ports (2)	0.44%	23 minutes ago		:	
☐	hive-metastore	6bbf4e020e3b	apache/hive:3.1.3	9083:9083 Show all ports (2)	0.25%	23 minutes ago		:	
☐	minio-init	49588c5b240b	minio/mc:latest		0%	23 minutes ago		:	
☐	airflow	78e7fe8a3e5a	docker-airflow	8083:8080 Show all ports (2)	2.36%	23 minutes ago		:	
☐	trino	1df0d4855fcd	trino/trino:latest	8082:8080 Show all ports (2)	1.99%	23 minutes ago		:	

Imagen 2: Gestión variables de entorno

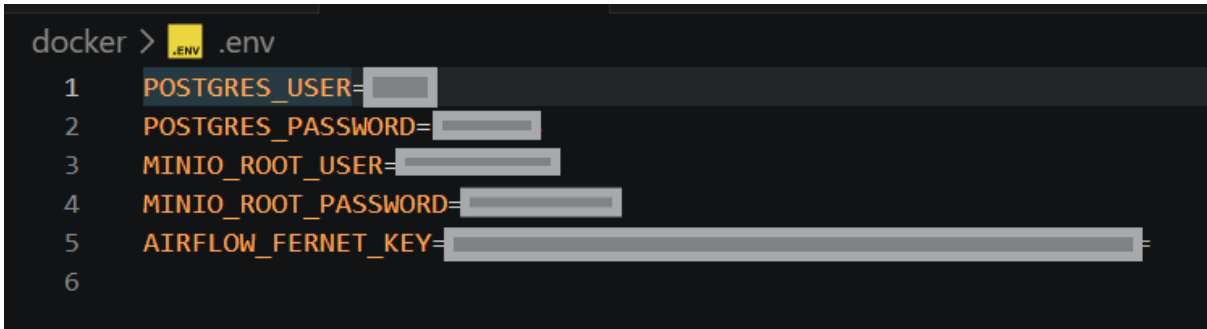


Imagen 3: Interfaz de Gestión - MinIO

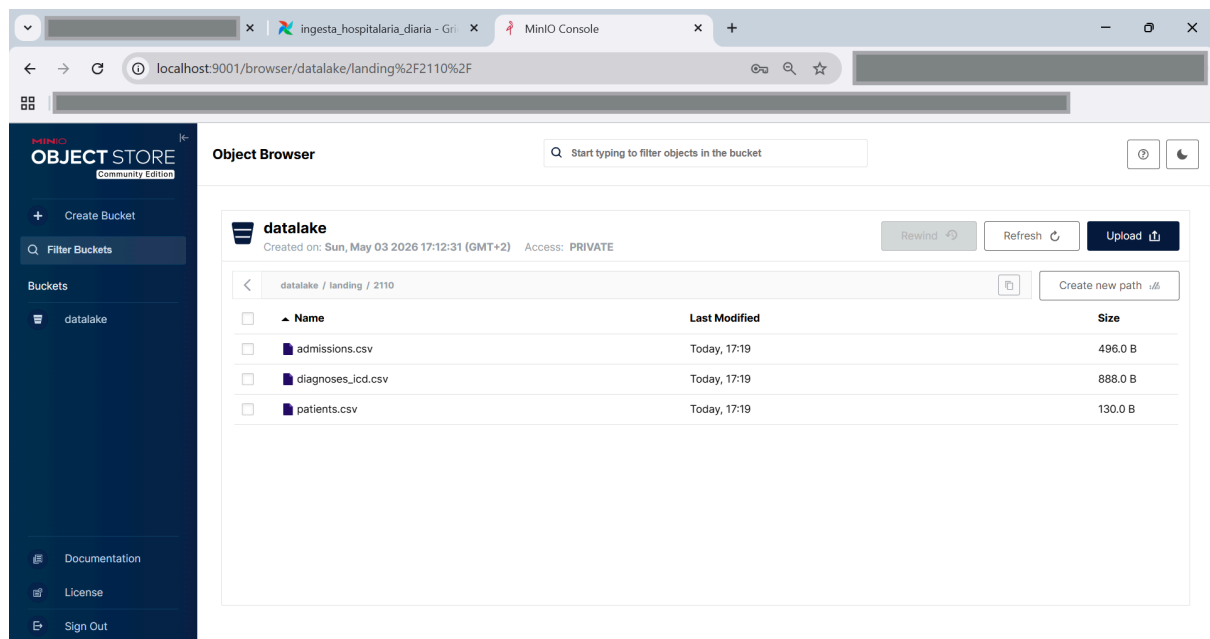


Imagen 3: Interfaz de Gestión - Airflow

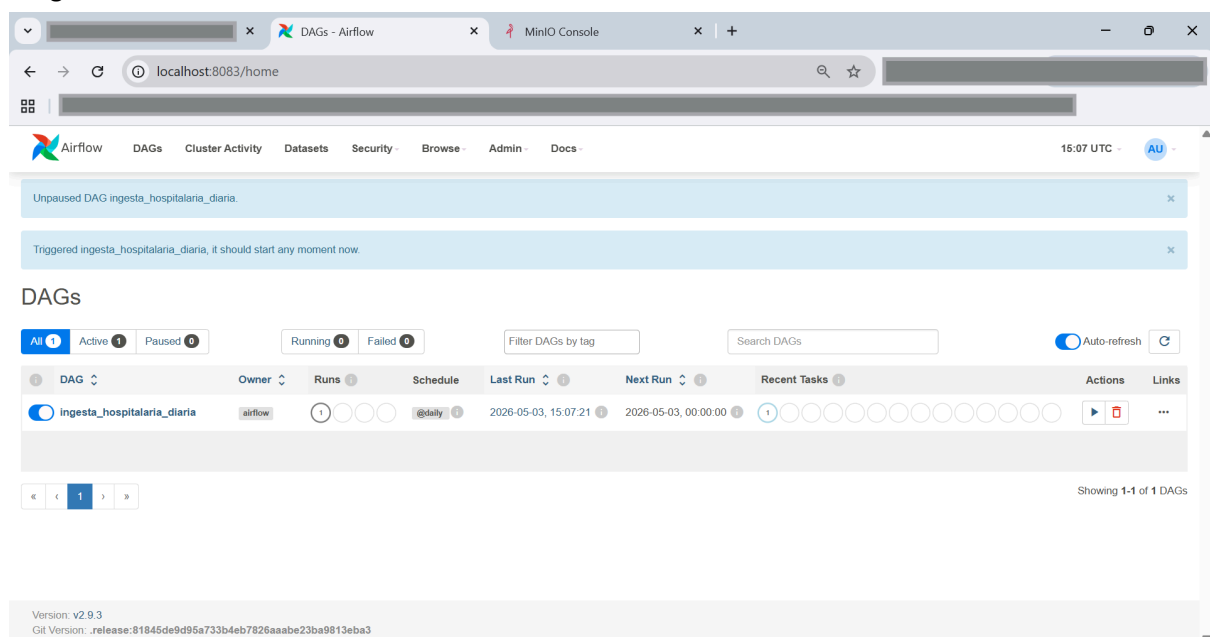


Imagen 5: Pipeline Graph View en Airflow completado

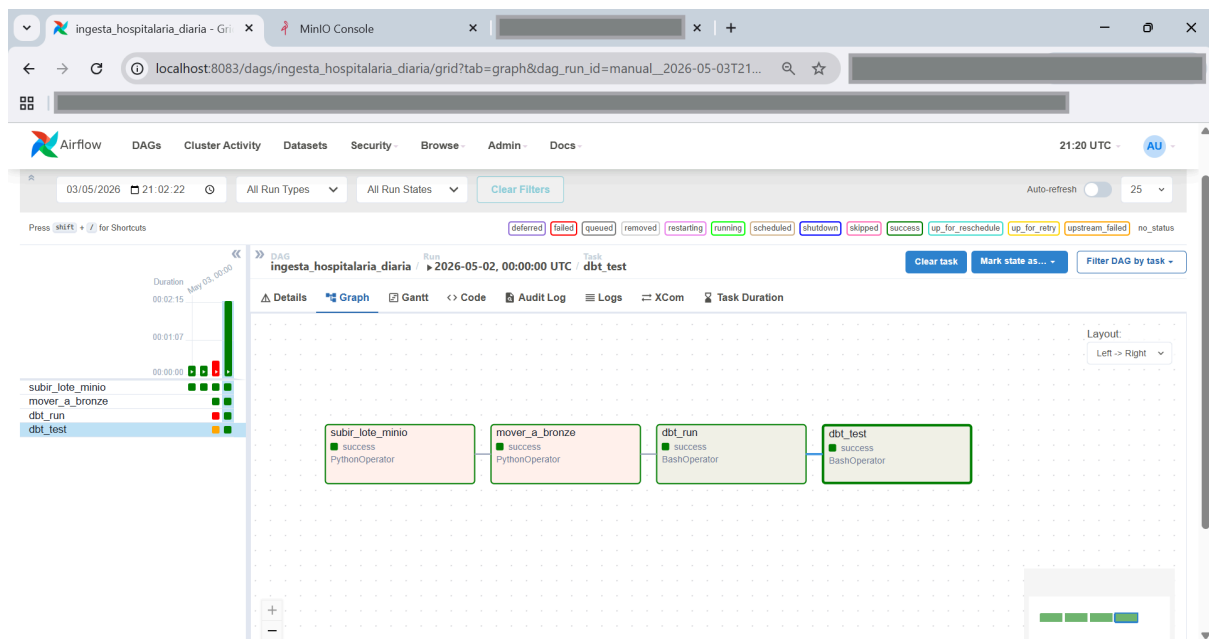


Imagen 6: Carga de datos en MiniO

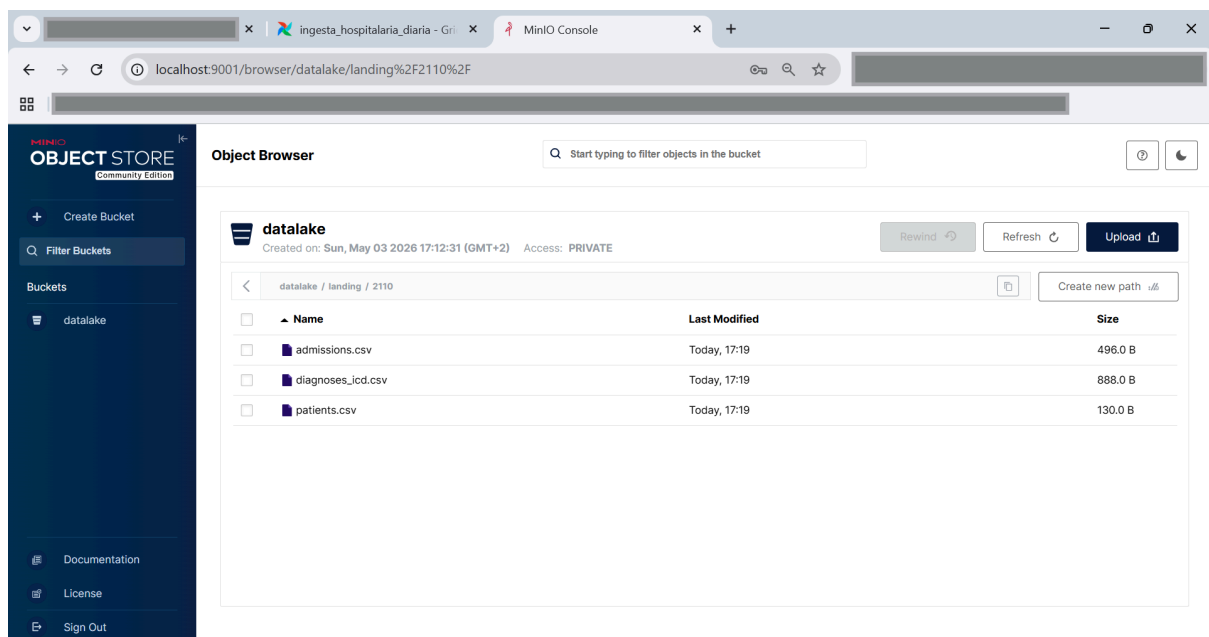


Imagen 7: Estructura y creación modelo person.sql a nivel **dbt_project**

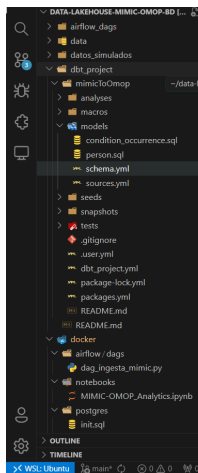


Imagen 8: Interfaz del Explorador de Objetos MiniIO: Bucket landing-zone

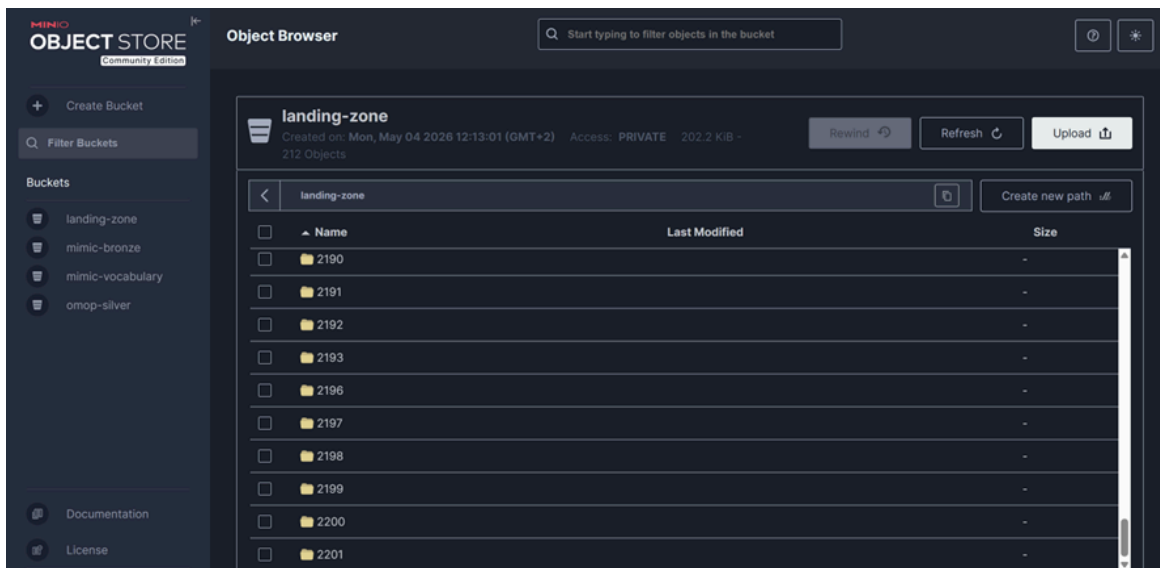


Imagen 9: Interfaz de MinIO: Visualización del Bucket omop-silver

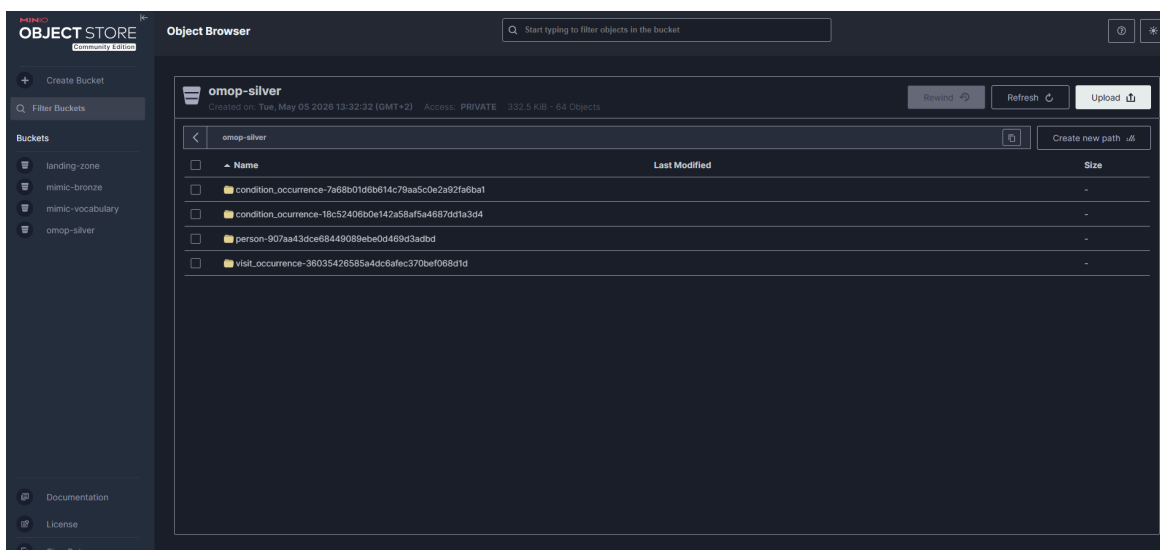
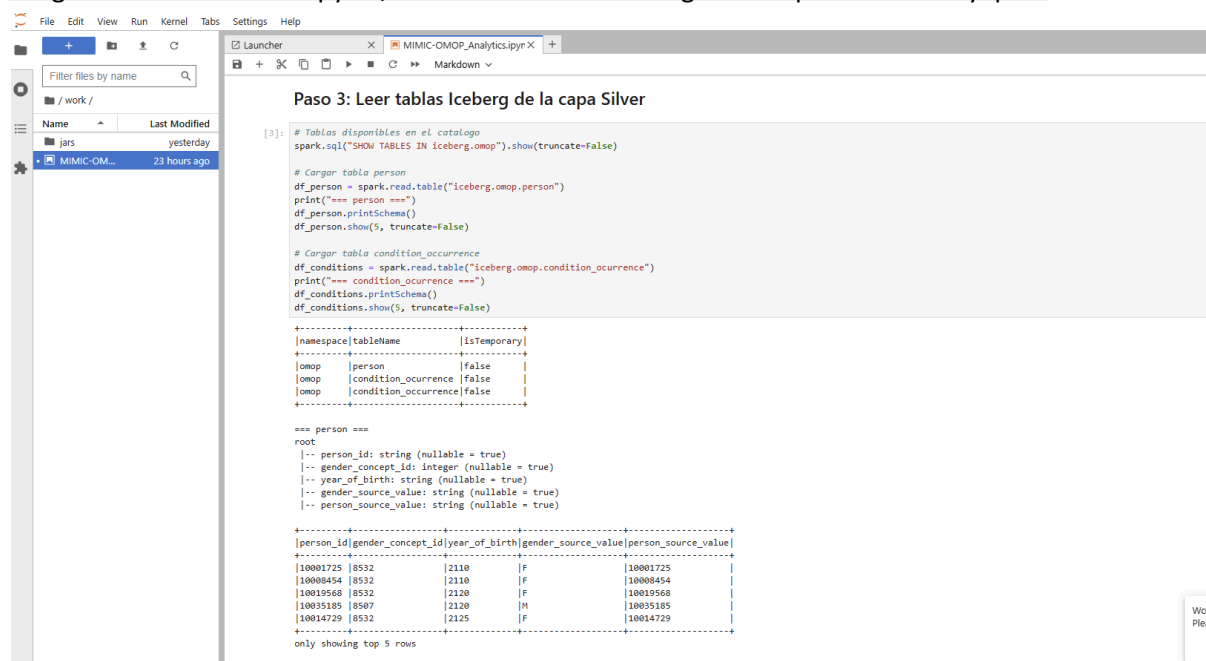


Imagen 9: Notebook de Jupyter; Lectura de Tablas Iceberg en la Capa Silver con PySpark



7. Reparto del trabajo

El reparto del trabajo para este trabajo lo hemos hecho por issues, hemos creado un proyecto en github (<https://github.com/pacorey16/data-lakehouse-mimic-omop-BD>) y hemos definido 10 issues correspondientes a todo el trabajo que debíamos realizar para la consecución del proyecto, estas issues han sido repartidas de manera equitativa y de la siguiente forma:

NOMBRES	ISSUES
Maria Luisa Rodriguez Cabrera	1,2,3
Paloma Bravo del Cid	4, 5, 6
Francisco Rey Garduño	7,8,9,10

Hemos llevado un control iterativo de las tareas defin

8. Conclusiones

- **Desafíos de integración:** La complejidad de configurar permisos y protocolos de red en Docker (S3 vs Hive).
- **Ventajas del Lakehouse:** Cómo dbt y Trino permiten tratar archivos planos como si fueran una base de datos relacional de alto rendimiento.
- **Escalabilidad:** El sistema está preparado para recibir millones de registros con sólo añadir más nodos o capacidad.
- **Desafíos de integración;** más de la mitad de los commits son correcciones de configuración, no de lógica.
- Al transformar MIMIC a OMOP, los datos se vuelven **interoperables:** cualquier herramienta estudio que hable OMOP (ATLAS, OHDSI, etc.) puede usar estos datos sin adaptación adicional.
- **El pipeline no es sólo infraestructura:** produce resultados clínicos concretos. Que la hipertensión (4019) sea el diagnóstico más frecuente y que hiperlipidemia + hipertensión sea la comorbilidad más común, es consistente con la literatura médica. Esto valida que los datos se transformaron correctamente
- **El diseño por capas** y el uso de formatos abiertos (**Iceberg, Parquet**) permiten **escalar** cada pieza de forma **independiente**. MinIO añade nodos de almacenamiento, Trino añade workers de cómputo, y los modelos dbt con merge incremental sólo procesan datos nuevos. No hay que rediseñar nada para pasar de miles a millones de registros.